

## **C# — 2027 Interview Questions**

### **10 Years Experience | Conceptual + Logical + Analytical + Scenario-Based**

#### **1. C# Modern Features**

1. What are the important features introduced in recent versions of C#?
  2. What is new in C# 14?
  3. What are field-backed properties in C#?
  4. What are extension blocks in C#?
  5. What are the recent improvements to `Span<T>` in C#?
  6. What are records and when would you use them?
  7. What is the difference between record, record class, and record struct?
  8. What are required members in C#?
  9. What is pattern matching in modern C#?
  10. What are primary constructors?
  11. What are collection expressions?
  12. What is the `..` range operator?
  13. What is the `^` index operator?
  14. What are raw string literals?
  15. What are file-scoped namespaces?
- 

#### **2. C# Fundamentals**

16. What is the difference between value types and reference types?
17. What happens internally when an object is created?
18. What is boxing and unboxing?
19. What is the difference between `var`, `dynamic`, and `object`?
20. What is the difference between `const`, `readonly`, and `static readonly`?
21. What is the difference between `==` and `Equals()`?
22. Why should `Equals()` and `GetHashCode()` be consistent?
23. What is the difference between `ref`, `out`, and `in`?
24. What is the difference between `class` and `struct`?

- 25. When would you choose a struct instead of a class?
  - 26. What is a readonly struct?
  - 27. What is a ref struct?
  - 28. Why can't a ref struct normally be stored on the heap?
  - 29. What is immutability?
  - 30. Why is immutability useful in concurrent applications?
- 

### **3. OOP & Advanced C#**

- 31. What are the four pillars of OOP?
  - 32. What is the difference between abstraction and encapsulation?
  - 33. What is the difference between abstract class and interface?
  - 34. Can an interface contain implementation?
  - 35. What is the difference between virtual, override, and new?
  - 36. What is method hiding?
  - 37. What is multiple inheritance and how does C# handle it?
  - 38. What is composition over inheritance?
  - 39. What is a sealed class?
  - 40. What is a static class?
  - 41. What is a partial class?
  - 42. What are nested classes?
  - 43. What are generic constraints?
  - 44. What are covariance and contravariance?
  - 45. What is the difference between compile-time and runtime polymorphism?
- 

### **4. Delegates, Events & Functional C#**

- 46. What is a delegate?
- 47. What is an event?
- 48. What is the difference between a delegate and an event?
- 49. What are Func, Action, and Predicate?

- 50. What are lambda expressions?
  - 51. What are expression trees?
  - 52. What is the difference between a delegate and an expression tree?
  - 53. What are extension methods?
  - 54. What are anonymous methods?
  - 55. When would you use events instead of callbacks?
- 

## 5. Collections & Generics

- 56. What is the difference between Array, List<T>, and LinkedList<T>?
  - 57. What is the difference between List<T> and HashSet<T>?
  - 58. What is the difference between Dictionary<TKey,TValue> and ConcurrentDictionary<TKey,TValue>?
  - 59. How does Dictionary<TKey,TValue> work internally?
  - 60. What happens when two dictionary keys have the same hash code?
  - 61. How does HashSet<T> work internally?
  - 62. What happens internally when a List<T> exceeds its capacity?
  - 63. What is the time complexity of lookup in a Dictionary?
  - 64. When would you use ConcurrentDictionary?
  - 65. What are generic constraints and why are they useful?
- 

## 6. LINQ — Modern Interview Questions

- 66. What is deferred execution?
- 67. What is immediate execution?
- 68. What is the difference between IEnumerable and IQueryable?
- 69. What is the difference between Select() and SelectMany()?
- 70. What is the difference between First() and Single()?
- 71. What is the difference between Any() and Count()?
- 72. What happens when you call ToList()?
- 73. Can the same LINQ query execute multiple times?
- 74. How can deferred execution cause a production bug?

- 75. How can LINQ create unnecessary database queries?
  - 76. When does LINQ become a performance problem?
  - 77. How would you optimize a LINQ query over millions of records?
- 

## **7. Async/Await — Very Important**

- 78. What is async/await?
  - 79. Does async/await create a new thread?
  - 80. What happens internally when execution reaches await?
  - 81. What is a Task?
  - 82. What is the difference between Task and Thread?
  - 83. What is ValueTask?
  - 84. When would you use ValueTask?
  - 85. What is Task.Run()?
  - 86. When should you avoid Task.Run()?
  - 87. What is Task.WhenAll()?
  - 88. What is Task.WhenAny()?
  - 89. What happens when one task inside Task.WhenAll() fails?
  - 90. How do you handle exceptions from multiple asynchronous operations?
  - 91. What is CancellationToken?
  - 92. How do you cancel an asynchronous operation?
  - 93. What is thread-pool starvation?
  - 94. Why can .Result and .Wait() be dangerous?
  - 95. Can asynchronous code still have race conditions?
- 

## **8. Multithreading & Concurrency**

- 96. What is multithreading?
- 97. What is concurrency?
- 98. What is parallelism?
- 99. What is the difference between concurrency and parallelism?

- 100. What is a race condition?
  - 101. What is a deadlock?
  - 102. What is thread safety?
  - 103. What is lock?
  - 104. What is Monitor?
  - 105. What is SemaphoreSlim?
  - 106. What is Mutex?
  - 107. What is the difference between lock and SemaphoreSlim?
  - 108. What is Interlocked?
  - 109. When would you use ConcurrentQueue<T>?
  - 110. When would you use ConcurrentDictionary<TKey,TValue>?
  - 111. How would you make a singleton thread-safe?
- 

## 9. Memory & Performance

- 112. How does Garbage Collection work in .NET?
- 113. What are Gen 0, Gen 1, and Gen 2?
- 114. What is the Large Object Heap?
- 115. What causes excessive garbage collection?
- 116. Can a managed C# application have a memory leak?
- 117. What is IDisposable?
- 118. What is the difference between Dispose() and a finalizer?
- 119. What happens internally when you use a using statement?
- 120. What is object pooling?
- 121. When would object pooling improve performance?
- 122. What is ArrayPool<T>?
- 123. What is Memory<T>?
- 124. What is Span<T>?
- 125. What is the difference between Span<T> and Memory<T>?
- 126. Why is Span<T> useful for high-performance code?

- 127. What is stack allocation with `stackalloc`?
  - 128. How would you identify unnecessary allocations in production?
- 

## 10. Logical Questions

- 129. Why can two variables referencing the same object modify each other's state?
  - 130. Why does modifying a copied value type not affect the original?
  - 131. Why can boxing hurt performance?
  - 132. Why can a struct sometimes perform worse than a class?
  - 133. Why are mutable structs generally discouraged?
  - 134. Why does a dictionary require correct `Equals()` and `GetHashCode()` implementations?
  - 135. Why can modifying a collection during enumeration throw an exception?
  - 136. Why can deferred execution produce unexpected results?
  - 137. Why can `ToList()` change the behavior of a LINQ query?
  - 138. Why doesn't `async` necessarily mean parallel execution?
  - 139. Why can `Task.Run()` make a Web API less scalable?
  - 140. Why can thread-pool starvation occur even when CPU isn't fully utilized?
- 

## 11. Analytical Questions

- 141. You have a method taking 5 seconds. How would you determine whether the bottleneck is CPU, I/O, memory, or locking?
- 142. A Dictionary lookup is unexpectedly slow. What would you investigate?
- 143. A LINQ operation is consuming huge amounts of memory. How would you analyze it?
- 144. You have millions of records in memory. Which collection would you choose and why?
- 145. You need very fast lookup by ID while preserving insertion order. What approach would you choose?
- 146. You have 10,000 independent operations. Would you use sequential execution, `Task.WhenAll()`, or parallelism?
- 147. How would you determine whether a method should be synchronous or asynchronous?
- 148. How would you identify unnecessary allocations?
- 149. How would you determine whether lock is causing a performance bottleneck?

- 150. How would you decide between class, struct, and record?
  - 151. How would you design a thread-safe in-memory cache?
  - 152. How would you handle concurrent modification of shared state?
- 

## 12. Real Production Scenarios

- 153. Your application's memory increases continuously after every deployment. How would you investigate it?
  - 154. Your API becomes extremely slow only when traffic increases. What C# issues would you investigate?
  - 155. Your application occasionally throws `NullReferenceException`, but you cannot reproduce it locally. How would you investigate?
  - 156. Two threads are modifying the same collection and the application occasionally fails. What could be happening?
  - 157. A developer has wrapped almost every method in `Task.Run()`. Would you approve this code?
  - 158. Your application experiences thread-pool starvation under high load. What would you investigate?
  - 159. A singleton service contains mutable state accessed by hundreds of requests. What problems can occur?
  - 160. A LINQ query works quickly for 1,000 records but becomes extremely slow for 10 million records. How would you investigate?
  - 161. Your application creates millions of short-lived objects and GC activity becomes very high. What would you consider changing?
  - 162. An asynchronous operation sometimes never completes. How would you investigate it?
  - 163. Several external calls must execute concurrently, but if one fails you need controlled cancellation of the others. How would you design it?
  - 164. You need to process a very large file without loading it entirely into memory. Which C# features would you consider?
  - 165. You need extremely high-throughput processing with minimal allocations. Which modern C#/.NET features would you investigate?
- 

## 13. 2027-Level Senior Questions

- 166. Which modern C# features have actually improved your production code, and why?

167. When would you use `Span<T>` instead of normal arrays or strings?
168. What trade-offs do you consider before using low-level performance optimizations?
169. When would `ValueTask` be better than `Task`, and when would it make the code worse?
170. How would you design highly concurrent C# code without relying heavily on locks?
171. How would you identify whether a performance problem requires code optimization or an architectural change?
172. How would you review another senior developer's asynchronous C# implementation?
173. What are the most common mistakes experienced developers make with `async/await`?
174. How would you design a reusable C# component that safely handles thousands of concurrent operations?
175. How would you approach refactoring a 10-year-old C# codebase without introducing regressions?
176. How do you balance clean, maintainable C# code against low-level performance optimization?
177. How would you diagnose CPU, memory, GC, and thread-pool problems in a production .NET application?
178. What C# features would you avoid using unnecessarily in high-throughput services?
179. How has modern C# changed the way you would design a new application compared with older C# versions?
180. If I give you an unfamiliar C# code snippet in an interview, how would you analyze its correctness, performance, thread safety, and maintainability?